EWB CLOUD LABS • AWS ARCHITECT ASSOCIATE • PROJECT 01

AWS VPC Peering: Private Network Connectivity Report

Designing non-overlapping VPC architectures, custom route tables, stateful security groups, and SSH ProxyJump administration

STUDENT NAME
Mohammed Asif Hudali

AWS ACCOUNT ID
`695313592048`

COMPLETION DATE
19 October 2026

AWS REGION
Sydney (`ap-southeast-2`)

PROJECT TAG
`Project = EWB-VPC-Peering`

STUDENT TAG
`Student = Mohammed Asif Hudali`

01 / Executive Summary & Project Mission

RUBRIC: ARCHITECTURE

This project establishes and verifies a private, direct network connection between two isolated Amazon Virtual Private Clouds (**VPC A** and **VPC B**) within AWS account `695313592048` in the Sydney (`ap-southeast-2`) Region. EWB operates an application server in VPC A (`ewb-app-a`) and a secondary workload in VPC B (`ewb-app-b`). Security posture mandates that both application servers remain strictly isolated from the public internet without public IPv4 addresses or NAT gateways, exchanging traffic solely across private IPv4 routes over an active AWS VPC Peering link (`pcx-0621164e8187e3c98`). A single public bastion host (`ewb-bastion`) deployed in VPC A's public subnet serves as the controlled administrative entry point via OpenSSH `ProxyJump` .

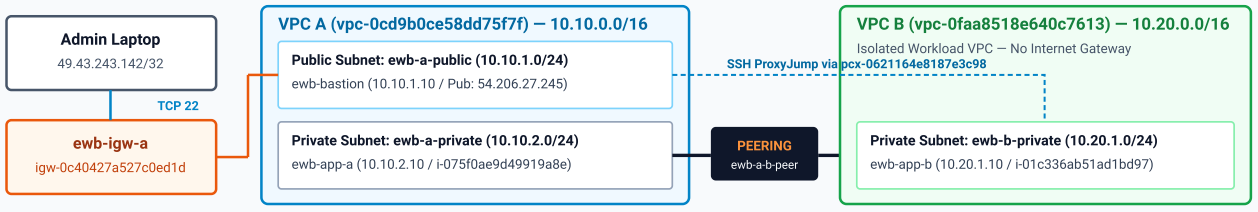


Figure 1.1: Deployed multi-VPC network topology in `ap-southeast-2` . Administrative SSH enters through `ewb-igw-a` (`igw-0c40427a527c0ed1d`) to `ewb-bastion` (`54.206.27.245` / `10.10.1.10`) and hops via SSH ProxyJump to `ewb-app-a` or `ewb-app-b` over `pcx-0621164e8187e3c98` .

02 / Non-Overlapping CIDR & Subnet Design Rationale

15 MARKS

A fundamental prerequisite of AWS VPC Peering is that the IPv4 CIDR blocks of the paired VPCs must not overlap. In this architecture, **VPC A** (`vpc-0cd9b0ce58dd75f7f`) is provisioned with `10.10.0.0/16` (65,536 IPv4 addresses: `10.10.0.0` – `10.10.255.255`) and **VPC B** (`vpc-0faa8518e640c7613`) is provisioned with `10.20.0.0/16` (`10.20.0.0` – `10.20.255.255`). Subnets are carved out on clean `/24` boundaries: `10.10.1.0/24` for VPC A Public, `10.10.2.0/24` for VPC A Private, and `10.20.1.0/24` for VPC B Private. Note that assigning non-overlapping subnet ranges would not cure an underlying VPC CIDR overlap; AWS validates uniqueness across all primary and secondary VPC CIDR blocks prior to establishing peering.

Key Architectural Constraints Verified During Build

- **Manual VPC Creation (`VPC only`):** Created without wizard defaults to maintain strict manual control over route tables, subnet associations, and internet gateways; no NAT Gateway was deployed, eliminating unnecessary hourly and data-processing costs.
- **Single Availability Zone Placement:** Subnets were aligned in Availability Zone `ap-southeast-2a` and launched using Amazon Linux 2023 on `t3.micro` x86 instances with `DeleteOnTermination` enabled on root EBS volumes.

03 / Complete AWS Resource Inventory & Actual IDs

CHECKPOINTS 01–03

All resources were provisioned in AWS Account `695313592048` (Sydney / `ap-southeast-2`) with consistent naming tags and static primary private IPv4 assignments:

Resource Name Tag	AWS Resource Type	Actual AWS Resource ID	VPC / Subnet / IPv4 CIDR / Address Details
ewb-vpc-a	VPC (Requester)	`vpc-0cd9b0ce58dd75f7f`	CIDR: `10.10.0.0/16` • Tenancy: Default • No IPv6
ewb-a-public	Public Subnet (VPC A)	`subnet-020420c23fdde2...`	CIDR: `10.10.1.0/24` • AZ: `ap-southeast-2a`
ewb-a-private	Private Subnet (VPC A)	`subnet-0eab8904f2e1b1...`	CIDR: `10.10.2.0/24` • Auto-assign Public IP: Disabled
ewb-igw-a	Internet Gateway	`igw-0c40427a527c0ed1d`	Attached to `ewb-vpc-a` (No IGW in VPC B)
ewb-vpc-b	VPC (Accepter)	`vpc-0faa8518e640c7613`	CIDR: `10.20.0.0/16` • Tenancy: Default • No IGW
ewb-b-private	Private Subnet (VPC B)	`subnet-03734726fa7d6a...`	CIDR: `10.20.1.0/24` • Auto-assign Public IP: Disabled
ewb-rt-a-public	Route Table (VPC A)	`rtb-06ad0c4d177537911`	Explicitly associated with `ewb-a-public`
ewb-rt-a-private	Route Table (VPC A)	`rtb-04ebe0b2e708cbf63`	Explicitly associated with `ewb-a-private`
ewb-rt-b-private	Route Table (VPC B)	`rtb-0986258f821b67d34`	Explicitly associated with `ewb-b-private`
ewb-a-b-peer	VPC Peering Connection	`pcx-0621164e8187e3c98`	State: **Active** • `10.10.0.0/16` <> `10.20.0.0/16`
ewb-sg-bastion	Security Group (VPC A)	`sg-0c55fbb4a837d7656`	Inbound TCP 22 from `49.43.243.142/32` (`sgr-0128f56409e68ebde`)
ewb-sg-app-a	Security Group (VPC A)	`sg-073f39f448b7c8c02`	Inbound TCP 22 (`10.10.1.10/32`), ICMP (`10.20.1.10/32`)
ewb-sg-app-b	Security Group (VPC B)	`sg-0ec58fbaf2e3f2128`	Inbound TCP 22 (`10.10.1.10/32`), ICMP (`10.10.2.10/32`)
ewb-bastion	EC2 Instance (AL2023)	`i-0dba867aac73b78d7`	Private IP: `10.10.1.10` • Public IP: `54.206.27.245`
ewb-app-a	EC2 Instance (AL2023)	`i-075f0ae9d49919a8e`	Private IP: `10.10.2.10` • Public IP: *None (Empty)*
ewb-app-b	EC2 Instance (AL2023)	`i-01c336ab51ad1bd97`	Private IP: `10.20.1.10` • Public IP: *None (Empty)*

04 / Route Table Associations & Bidirectional Peering Routes

CHECKPOINT 04 • 40 MARKS

Activating `ewb-a-b-peer` (`pcx-0621164e8187e3c98`) creates the logical link, but network packets only traverse peering when both forward and return routes exist in the explicitly associated subnet route tables:

1. ewb-rt-a-private (rtb-04ebe0b2e708cbf63)

Local Route: `10.10.0.0/16` → `local` (Intra-VPC traffic, including Bastion to App-A).

2. ewb-rt-b-private (rtb-0986258f821b67d34)

Local Route: `10.20.0.0/16` → `local` (Existing intra-VPC B route).

05 / Host-Scoped Security Groups & SSH ProxyJump Setup

RUBRIC: SECURITY • 20 MARKS

All three security groups enforce strict `/32` single-host source restrictions rather than broad subnet CIDRs or `0.0.0.0/0`. Because AWS security groups are **stateful**, return traffic for permitted inbound TCP/ICMP flows is automatically allowed without custom outbound rules.

Security Group & ID	Rule ID	Protocol / Port	Allowed Source CIDR	Operational Purpose
ewb-sg-bastion (`sg-0c55fbb4a837d7656`)	`sgr-0128f56409e68ebde`	SSH / TCP 22	`49.43.243.142/32`	SSH login from admin laptop
ewb-sg-app-a (`sg-073f39f448b7c8c02`)	`sgr-028741977950681df`	SSH / TCP 22	`10.10.1.10/32`	SSH via `ewb-bastion` private IP
ewb-sg-app-a (`sg-073f39f448b7c8c02`)	`sgr-09f90bd172a5138ee`	All ICMP - IPv4	`10.20.1.10/32`	ICMP Ping from `ewb-app-b`
ewb-sg-app-b (`sg-0ec58fbaf2e3f2128`)	`sgr-022f0fe727ba496e9`	SSH / TCP 22	`10.10.1.10/32`	SSH via `ewb-bastion` over peering
ewb-sg-app-b (`sg-0ec58fbaf2e3f2128`)	`sgr-03614fc75dd830443`	All ICMP - IPv4	`10.10.2.10/32`	ICMP Ping from `ewb-app-a`

POWERSHELL: C:\Users\hudal\.ssh\config (icacls locked)

```
Host ewb-bastion HostName 54.206.27.245 User ec2-user IdentityFile ~/.ssh/ewb-lab.pem IdentitiesOnly yes Host ewb-app-a HostName 10.10.2.10 User ec2-user IdentityFile ~/.ssh/ewb-lab.pem IdentitiesOnly yes ProxyJump ewb-bastion Host ewb-app-b HostName 10.20.1.10 User ec2-user IdentityFile ~/.ssh/ewb-lab.pem IdentitiesOnly yes ProxyJump ewb-bastion
```

Key Custody & Forward/Return Path Mechanics

Zero Private Keys on EC2: Using `icacls` in Windows PowerShell to restrict `C:\Users\hudal\.ssh\ewb-lab.pem` and configuring `ProxyJump ewb-bastion` keeps the private key strictly on the local laptop.

Forward vs. Return Path: When `ewb-app-a` (`10.10.2.10`) pings `ewb-app-b` (`10.20.1.10`), the *forward packet* uses `rtb-04ebe0b2e708cbf63` (`10.20.0.0/16` → `pcx-0621164e8187e3c98`) and matches rule `sgr-03614fc75dd830443` . The *Echo Reply* uses `rtb-0986258f821b67d34` (`10.10.0.0/16` → `pcx-0621164e8187e3c98`) and returns statefully.

06 / Recorded Terminal Output: Baseline & Post-Peering Tests

TESTS 01–05 • 15 MARKS

TEST 01 / BASELINE (Host: ewb-app-a • Before Peering & Route Configuration) — **100% PACKET LOSS (3157ms)**

```
[ec2-user@ip-10-10-2-10 ~]$ hostname && hostname -I ip-10-10-2-10.ap-southeast-2.compute.internal 10.10.2.10 [ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10 PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data. --- 10.20.1.10 ping statistics --- 4 packets transmitted, 0 received, 100% packet loss, time 3157ms
```

TEST 02 / FORWARD PATH (ewb-app-a → 10.20.1.10) — **0% LOSS**

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10 PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data. 64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.685
```

TEST 03 & 04 / REVERSE PATH & SSH (ewb-app-b) — **0% LOSS**

```
PS C:\Users\hudal> ssh ewb-app-b [ec2-user@ip-10-20-1-10 ~]$ ping -c 4 10.10.2.10 PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data. 64 bytes from 10.10.2.10:
```

07 / Lab Execution Screenshots: Console & Baseline Route Tables EVIDENCE GALLERY I

Visual verification of AWS account setup (`695313592048` in `ap-southeast-2`) and Checkpoint 01 initial route tables prior to adding peering routes:

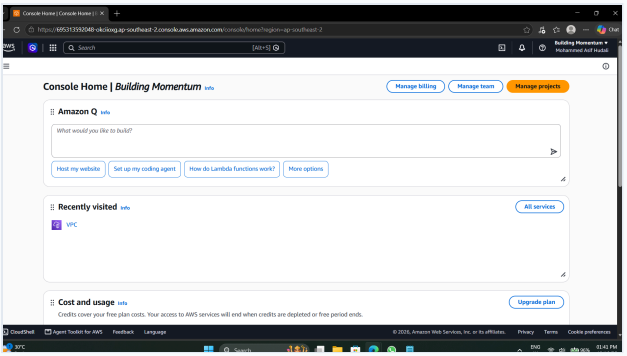


Fig 7.1 (Account & Region): AWS Console Home logged in as *Mohammed Asif Hudali* (`695313592048`) in Sydney (`ap-southeast-2`).

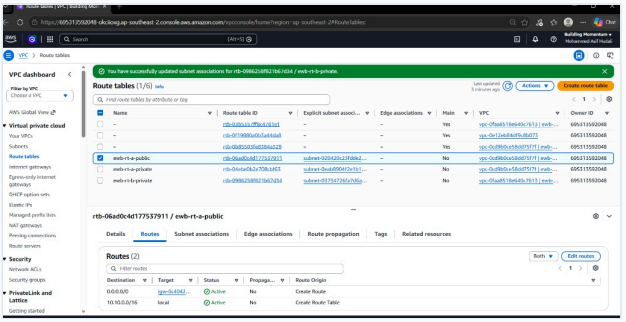


Fig 7.2 (Checkpoint 01 — Public RT): `ewb-rt-a-public` (`rtb-06ad0c4d177537911`) with `10.10.0.0/16 → local` and `0.0.0.0/0 → igw-0c40427a527c0ed1d` .

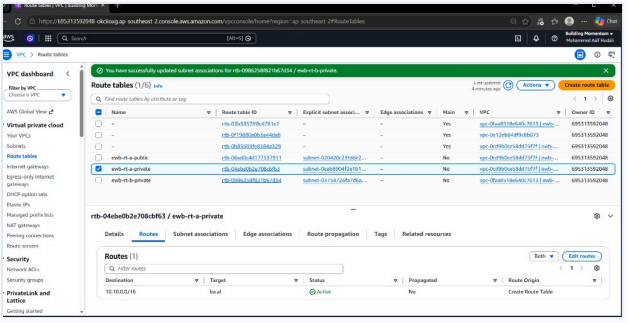


Fig 7.3 (Checkpoint 01 — Private RT A): `ewb-rt-a-private` (`rtb-04ebe0b2e708cbf63`) associated with `subnet-0eab8904f2e1b1...` containing only `10.10.0.0/16 → local` .

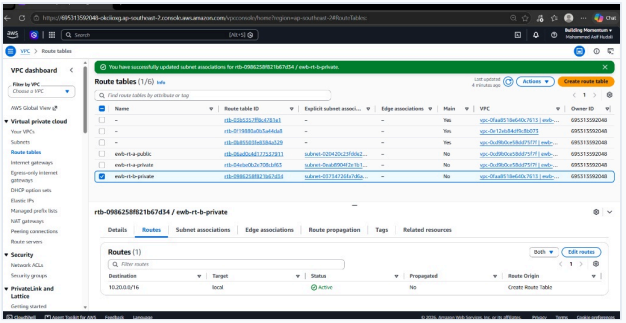


Fig 7.4 (Checkpoint 01 — Private RT B): `ewb-rt-b-private` (`rtb-0986258f821b67d34`) associated with `subnet-03734726fa7d6a...` containing only `10.20.0.0/16 → local` .

Checkpoint 01 Verification Summary

As captured in Figures 7.2–7.4, all three custom route tables (`ewb-rt-a-public` , `ewb-rt-a-private` , and `ewb-rt-b-private`) were explicitly associated with their respective subnets before launching EC2 instances. Neither private route table includes a `0.0.0.0/0` route, guaranteeing full isolation of `ewb-app-a` and `ewb-app-b` from direct internet traffic.

08 / Lab Execution Screenshots: Security Groups & EC2 Isolation

EVIDENCE GALLERY II

Console evidence of host-scoped `/32` inbound security group rules across VPC A and VPC B, followed by EC2 status checks and public IP isolation:

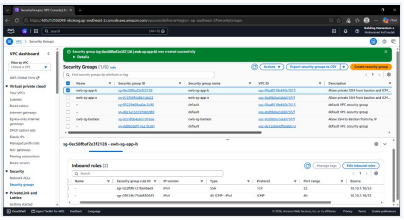


Fig 8.1 (`ewb-sg-app-b`): `sg-0ec58fbaf2e3f2128` allowing TCP 22 from `10.10.1.10/32` and ICMP from `10.10.2.10/32` .

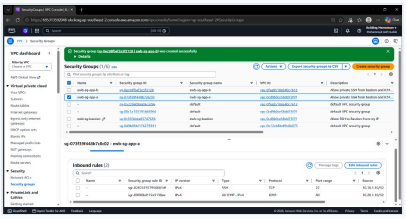


Fig 8.2 (`ewb-sg-app-a`): `sg-073f39f448b7c8c02` allowing TCP 22 from `10.10.1.10/32` and ICMP from `10.20.1.10/32` .

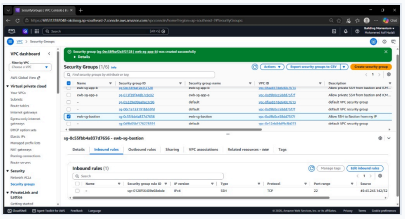


Fig 8.3 (`ewb-sg-bastion`): `sg-0c55fbb4a837d7656` allowing TCP 22 strictly from `49.43.243.142/32` .

09 / Checkpoint 02 & Test 05: EC2 Launch & Public IP Isolation Proof

EC2 INSTANCES

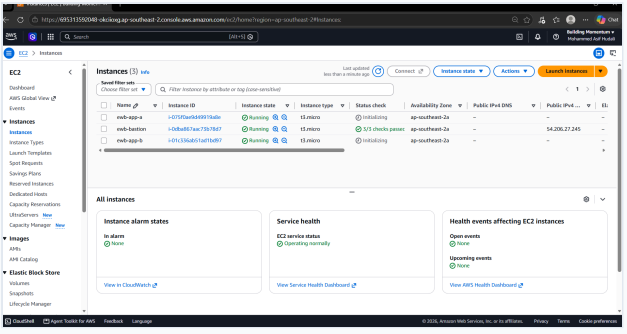


Fig 9.1 (EC2 Launch): `ewb-app-a` (`i-075f0ae9d49919a8e`), `ewb-bastion` (`i-0dba867aac73b78d7`), and `ewb-app-b` (`i-01c336ab51ad1bd97`) initializing in `ap-southeast-2a` .

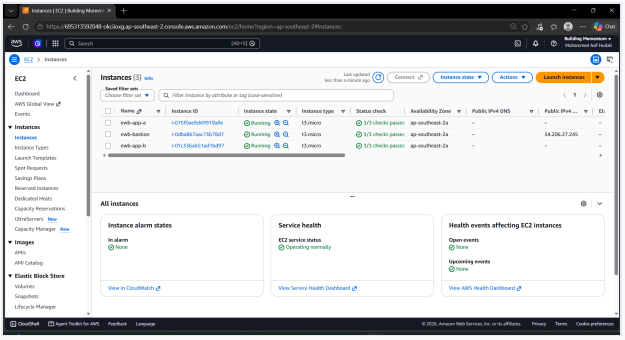


Fig 9.2 (Test 05 Isolation Proof): All three `t3.micro` instances passing `3/3 checks` . Only `ewb-bastion` has a Public IPv4 (`54.206.27.245`); both private hosts show `–` .

Checkpoint 02 & Test 05 Verification

Figure 9.2 confirms that `ewb-app-a` and `ewb-app-b` have empty Public IPv4 DNS and Public IPv4 address fields (`–`), proving neither workload can be reached directly from the internet.

10 / Lab Screenshots: SSH ProxyJump & Test 01 Baseline Failure EVIDENCE GALLERY III

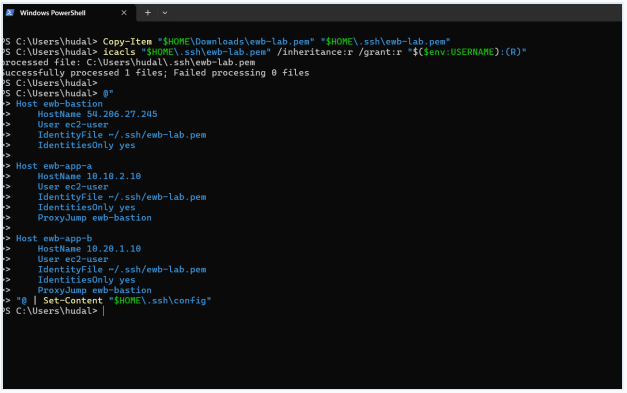


Fig 10.1 (SSH Key & Config): Locking `ewb-lab.pem` via `icacls` and writing `C:\Users\hudal\.ssh\config` with `ProxyJump ewb-bastion`.

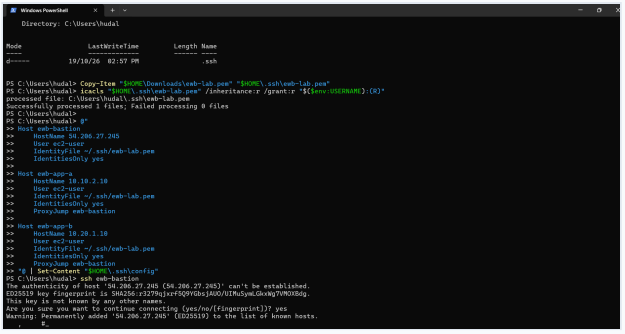


Fig 10.2 (Bastion SSH Login): Connecting from local PowerShell to `ewb-bastion` (`54.206.27.245`) using `ewb-lab.pem`.

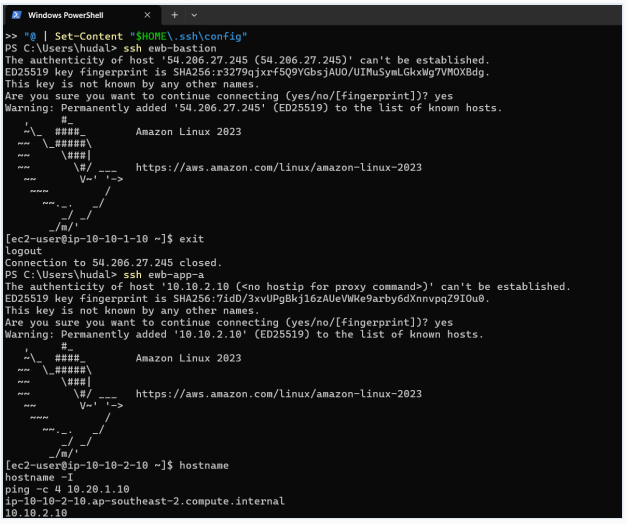


Fig 10.3 (ProxyJump to App-A): Exiting Bastion and connecting directly to `ewb-app-a` (`10.10.2.10`) over VPC A's local route.

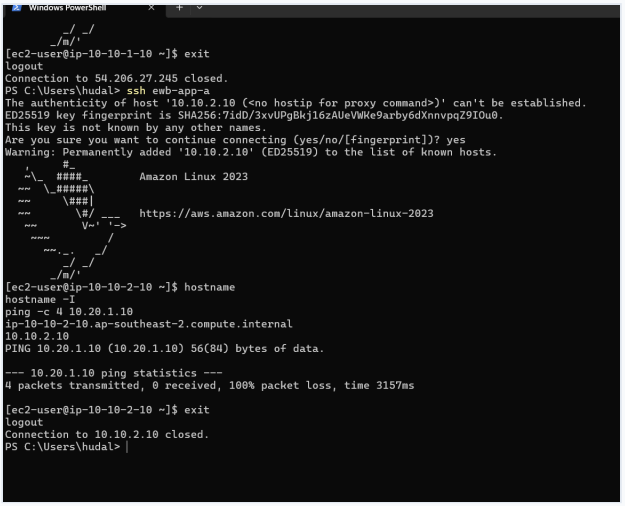


Fig 10.4 (Test 01 Baseline Failure): Running `ping -c 4 10.20.1.10` on `ewb-app-a` before peering yields `100% packet loss`.

11 / Lab Screenshots: Active Peering & Updated Route Tables CHECKPOINTS 03–04

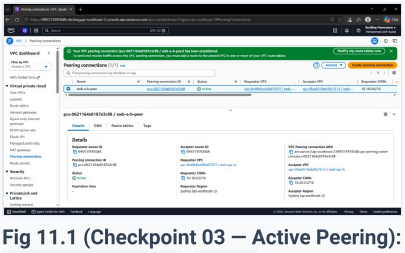


Fig 11.1 (Checkpoint 03 — Active Peering): `ewb-a-b-peer` (`pcx-0621164e8187e3c98`) Active between VPC A and VPC B.

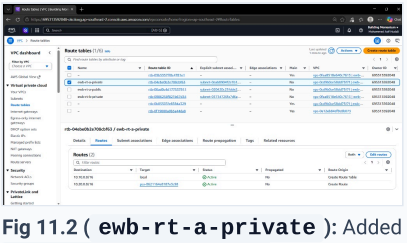


Fig 11.2 (`ewb-rt-a-private`): Added `10.20.0.0/16 → pcx-0621164e8187e3c98` forward route.

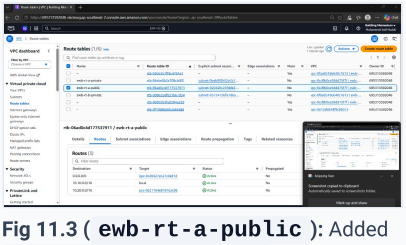


Fig 11.3 (`ewb-rt-a-public`): Added `10.20.0.0/16 → pcx-0621164e8187e3c98` for Bastion ProxyJump.

12 / Lab Screenshots: Return Route & Bidirectional Ping Success

TESTS 02–04

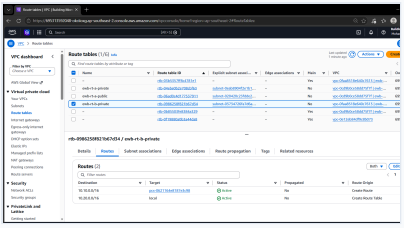


Fig 12.1 (`ewb-rt-b-private` Return Route): `rtb-0986258f821b67d34` configured with `10.10.0.0/16 → pcx-0621164e8187e3c98` (confirmed in `151543.jpg`).

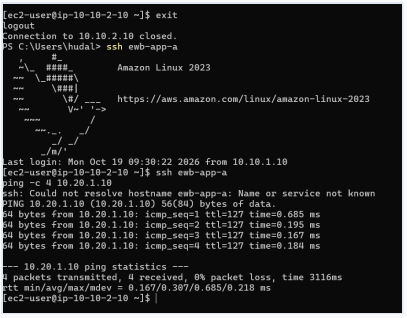


Fig 12.2 (Test 02 Forward Ping): `ewb-app-a` pinging `10.20.1.10` with `0% packet loss` (`rtt avg 0.307 ms`).

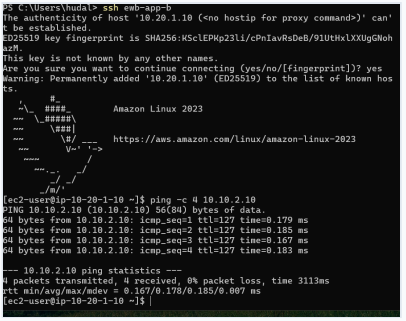


Fig 12.3 (Test 03 & 04 Reverse Ping & SSH): SSH ProxyJump into `ewb-app-b` (`10.20.1.10`) and ping to `10.10.2.10` (`0% loss` , `avg 0.178 ms`).

13 / Controlled Failure, Technical Viva Defense & Teardown

WORKBOOK PAGES 11–14

Controlled Failure & Recovery: Temporarily removing rule `sgr-03614fc75dd830443` (inbound ICMP from `10.10.2.10/32`) on `ewb-sg-app-b` caused `ping -c 4 10.20.1.10` from `ewb-app-a` to drop to `100% packet loss` while SSH remained active. Restoring the rule immediately recovered `0% packet loss` .

1. Why is the return route needed in `ewb-rt-b-private` (`rtb-0986258f821b67d34`)?

ICMP Echo Requests/Replies and TCP handshakes are bidirectional. Even when `ewb-rt-a-private` routes packets to `10.20.1.10` via `pcx-0621164e8187e3c98` , `ewb-app-b` must consult its own subnet's route table (`ewb-rt-b-private`) to send the reply back to `10.10.0.0/16` .

2. Why must `ewb-sg-app-b` allow SSH (TCP 22) from the bastion's private IP (`10.10.1.10/32`) rather than `49.43.243.142/32` ?

With OpenSSH `ProxyJump` , `ewb-bastion` opens a new TCP connection across the peering connection to `10.20.1.10:22` . Because peering routes private IPv4 traffic, `ewb-app-b` sees the bastion's primary private IP (`10.10.1.10`) as the source.

3. Why does an `Active` VPC Peering status not guarantee a successful `ping` , and can VPC B use `ewb-igw-a` ?

An `Active` peering connection only builds the link; traffic still requires forward/return route table entries and inbound security group rules. Furthermore, VPC peering is non-transitive and does not support edge-to-edge routing, so VPC B cannot access the internet through VPC A's Internet Gateway (`igw-0c40427a527c0ed1d`).

> **Responsible Teardown & Key Reflections**
>
> **Cleanup Completed:** Terminated `ewb-bastion` (`i-0dba867aac73b78d7`), `ewb-app-a` (`i-075f0ae9d49919a8e`), and `ewb-app-b` (`i-01c336ab51ad1bd97`); deleted peering routes and `pcx-0621164e8187e3c98` ; removed custom security groups, subnets, route tables, detached/deleted `igw-0c40427a527c0ed1d` , and deleted both VPCs (`vpc-0cd9b0ce58dd75f7f` and `vpc-0faa8518e640c7613`). Completing this lab demonstrated how longest-prefix match (`10.20.0.0/16` overriding `0.0.0.0/0` on `ewb-rt-a-public`) enables seamless cross-VPC SSH ProxyJump administration.